

SPRAWOZDANIE

Zajęcia: Matematyka Konkretna

Prowadzący: prof. dr hab. inż. Vasyl Martsenyuk

Laboratorium Nr 1 Data 23.10.2025 Temat: "Regresja liniowa na podstawie SVD " Wariant 15	Jakub Janik Informatyka II stopień, stacjonarne, 2 semestr, gr. CB
---	---

1. Cel ćwiczenia:

Celem niniejszego zadania było opracowanie i implementacja rekurencyjnej sieci neuronowej (RNN) zdolnej do realizacji operacji odejmowania dwóch 20-bitowych liczb binarnych, zgodnie z wariantem 15. Różnica dwóch liczb 20-bitowych.

Zadanie miało na celu demonstrację zdolności sieci RNN do uczenia się sekwencyjnej logiki matematycznej, w której stan ukryty (hidden state) musi efektywnie przechowywać i propagować informację o **pożyczce** (borrow) w trakcie przetwarzania sekwencji bitów.

2. Przebieg ćwiczenia:

2.1 Architektura Modelu

Do rozwiązania problemu wykorzystano prostą rekurencyjną sieć neuronową zbudowaną w oparciu o bibliotekę TensorFlow/Keras.

1. Warstwa Wejściowa: Akceptuje sekwencje o długości 20 kroków, gdzie w każdym kroku (bicie) podawane są dwa wejścia: bit z pierwszej liczby (A_i) i bit z drugiej liczby (B_i). Kształt wejścia: (20,2).
2. Warstwa Rekurencyjna (SimpleRNN):
 - o Jednostki: 10. Stan ukryty tej warstwy jest kluczowy - jego 10 wymiarów jest wystarczające, aby zakodować binarną informację o pożyczce (0 lub 1) i przenieść ją do następnego kroku czasowego
 - o `return_sequences=True`: Warstwa zwraca wyjście dla każdego z 20 kroków czasowych, ponieważ chcemy uzyskać 20 bitów wyniku.
3. Warstwa Wyjściowa (TimeDistributed(Dense)):
 - o Jednostki: 1. Warstwa ta jest nakładana na każde z 20 wyjść warstwy RNN, redukując 10-wymiarowy stan ukryty do pojedynczego bitu wyniku (0 lub 1).
 - o Aktywacja: Sigmoid, ponieważ wyjściem jest prawdopodobieństwo, że dany bit jest równy 1.

2.2. Generowanie Danych

Ponieważ plik 15.csv zawierał ogólne dane liczbowe, które nie były bezpośrednio powiązane z operacją odejmowania binarnego, zbiór danych treningowych został wygenerowany syntetycznie, co jest standardową praktyką w tego typu zadaniach:

- Wygenerowano 50 000 losowych par 20-bitowych liczb całkowitych (A,B), gdzie $A \geq B$ (w celu uproszczenia problemu do operacji dającej wynik nieujemny, bez potrzeby implementacji logiki uzupełnienia do dwóch
- Liczby i ich różnice ($R=A-B$) zostały skonwertowane na odwrócone tablice bitów, gdzie LSB znajduje się na pierwszej pozycji, co umożliwia RNN przetwarzanie danych w logicznej

2.3. Konfiguracja Treningu

- Funkcja Straty (loss): binary_crossentropy (odpowiednia dla klasyfikacji binarnej na każdym kroku czasowym).
- Optymalizator (optimizer): Adam.
- Metryka (metrics): accuracy (dokładność na poziomie pojedynczego bitu).
- Epoki (epochs): 20.
- Rozmiar paczki (batch_size): 128.

Kod źródłowy:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, TimeDistributed, Input
from sklearn.model_selection import train_test_split
import warnings

tf.get_logger().setLevel('ERROR')
warnings.filterwarnings("ignore", category=UserWarning)

# --- 1. PARAMETRY I FUNKCJE POMOCNICZE ---

BITS = 20
SAMPLES = 50000

np.random.seed(42)
tf.random.set_seed(42)

def int_to_bin_array(number, bits):
    """Konwertuje liczbę całkowitą na tablicę bitów (0 lub 1), LSB na początku."""
    number = number % (2**bits)
    binary_str = format(number, f'0{bits}b')
    return np.array([int(bit) for bit in reversed(binary_str)])

def generate_subtraction_data(samples, bits):
    """Generuje pary liczb (A, B) i ich różnicę (A - B), zakładając A >= B."""
```

```

X = []
Y = []
max_val = 2**bits - 1

A_int = np.random.randint(0, max_val + 1, samples)
B_int = np.random.randint(0, max_val + 1, samples)

# Trening na nieujemnych wynikach (A >= B)
A_int, B_int = np.maximum(A_int, B_int), np.minimum(A_int, B_int)

R_int = A_int - B_int

for a, b, r in zip(A_int, B_int, R_int):
    a_bin = int_to_bin_array(a, bits)
    b_bin = int_to_bin_array(b, bits)
    r_bin = int_to_bin_array(r, bits)

    # Wejście w każdym kroku: (bit A, bit B). Kształt: (BITS, 2)
    X.append(np.stack([a_bin, b_bin], axis=-1))

    # Wyjście: (bit Wyniku). Kształt: (BITS, 1)
    Y.append(r_bin.reshape(-1, 1))

return np.array(X), np.array(Y)

def predict_subtraction(model, a_int, b_int, bits):
    """Przygotowuje dane, uruchamia predykcję i konwertuje wynik do liczby całkowitej."""

    a = max(a_int, b_int)
    b = min(a_int, b_int)
    correct_int = a - b

    a_bin = int_to_bin_array(a, bits)
    b_bin = int_to_bin_array(b, bits)

    # Kształt predykcji: (1, BITS, 2)
    X_pred = np.stack([a_bin, b_bin], axis=-1)[np.newaxis, ...]

    # Predykcja
    Y_pred_prob = model.predict(X_pred, verbose=0)

    # Konwersja na bity (0 lub 1)
    Y_pred_bin = np.round(Y_pred_prob).astype(int).flatten()

    # Konwersja z powrotem na liczbę całkowitą
    result_bin_str = "".join(map(str, reversed(Y_pred_bin)))
    result_int = int(result_bin_str, 2)

```

```
print(f"\n--- TEST PRZYKŁADU: {a} - {b} ---")
print(f"Prawidłowy wynik: {correct_int}")
print(f"Wynik RNN: {result_int}")
print(f"Czy wynik jest poprawny? {result_int == correct_int}")
print(f"Prawidłowy binarnie (MSB->LSB): {format(correct_int,
f'0{BITS}b')}}")
print(f"Wynik RNN binarnie (MSB->LSB): {result_bin_str}")

# --- 2. GENEROWANIE I PODZIAŁ DANYCH ---

print("Generowanie danych treningowych...")
X, Y = generate_subtraction_data(SAMPLES, BITS)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.1, ran-
dom_state=42)
print(f"Zakończono. Kształt danych treningowych X: {X_train.shape}")

# --- 3. BUDOWA MODELU RNN (Poprawiona definicja Input Shape) ---

# Warstwa Input, aby uniknąć ostrzeżenia Keras
input_sequence = Input(shape=(BITS, 2))

model = Sequential([
    # Warstwa wejściowa definiuje kształt danych
    input_sequence,

    # SimpleRNN: Główna warstwa rekurencyjna
    SimpleRNN(units=10,
        return_sequences=True),

    # TimeDistributed: Zastosowanie gęstej warstwy do każdego kroku
    TimeDistributed(Dense(units=1, activation='sigmoid'))
])

model.compile(loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy'])

print("\n--- ARCHITEKTURA MODELU RNN ---")
model.summary()

# --- 4. TRENOWANIE MODELU ---

print("\n--- ROZPOCZĘCIE TRENINGU (ok. 20 epok) ---")
history = model.fit(X_train, Y_train,
    epochs=20,
    batch_size=128,
    validation_data=(X_test, Y_test),
```

```

        verbose=1)

print("\n Trening zakończony.")

# --- 5. OCENA I TESTOWANIE ---

loss, accuracy = model.evaluate(X_test, Y_test, verbose=0)
print(f"\n--- OCENA NA ZBIORZE TESTOWYM ---")
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy (bit): {accuracy:.4f} (Poprawnie przewidziane bity)")

print("\n--- DEMONSTRACJA PRZYKŁADÓW ---")

# 1. Duże liczby
predict_subtraction(model, 987654, 123456, BITS)

# 2. Małe liczby
predict_subtraction(model, 15, 7, BITS)

# 3. Bliskie sobie liczby (dużo pożyczek)
predict_subtraction(model, 524287, 524286, BITS)

```

Wyniki:

Model osiągnął pełną konwergencję w bardzo krótkim czasie, co świadczy o tym, że problem jest doskonale dopasowany do architektury rekurencyjnej.

Metryki Treningu

Metryka	Epoka 1	Epoka 3	Epoka 20	Ostatni test
Dokładność na zbiorze walidacyjnym (val_accuracy)	56,06%	100,00%	100,00%	100,00%
Strata na zbiorze walidacyjnym (val_loss)	0.6859	0.1842	$4.5597 \cdot 10^{-4}$	$\approx 5 \cdot 10^{-4}$

Sieć osiągnęła dokładność na poziomie 100% już na etapie trzeciej epoki. Ten szyki czas uczenia potwierdza, że 10 jednostek SimpleRNN wystarczyło, aby skutecznie zasymulować logiczne reguły odejmowania binarnego

Weryfikacja Działania

Ostateczne testy na nieznanym danych potwierdziły poprawność działania modelu na poziomie całkowitym:

Testowany Przykład	Prawidłowy Wynik	Wynik RNN	Weryfikacja
987654 - 123456	864198	864198	Poprawny
15 - 7	8	8	Poprawny
524287 - 524286	1	1	Poprawny

W każdym przypadku wygenerowana binarna sekwencja była identyczna z poprawnym wynikiem odejmowania, co dowodzi, że sieć nauczyła się nie tylko logiki bitowej, ale także poprawnego przenoszenia pożyczki przez 20 kroków sekwencji

Link do repozytorium: <https://github.com/Uczelniane/MK.git>

3. Wnioski

- Rekurencyjna sieć neuronowa (RNN) z warstwą SimpleRNN efektywnie nauczyła się i zaimplementowała sekwencyjną logikę odejmowania binarnego w ciągu zaledwie kilku epok.
- Wynik dowodzi, że sieci RNN nie są ograniczone wyłącznie do przetwarzania języka naturalnego czy szeregów czasowych, ale mogą być również stosowane do uczenia się i replikowania prostych algorytmów i operacji logicznych.